New York City Taxi Fare Prediction

Arijit Guchhait

Research Objectives

For each key in the test set, you must predict a value for the fare_amount variable. The evaluation metric for this competition is the root mean-squared error or RMSE.

Data Wrangling

Loading library & Data

- train.csv - Input features and target fare_amount values for the training set. Only 01 Million rows are seleted out of 55 M due to memory restriction on local machine.
- test.csv - Input features for the test set (about 10K rows). Goal is to predict fare_amount for each row.
- sample_submission.csv - a sample submission file in the correct format (columns key and fare_amount). This file 'predicts' fare_amount to be $11.35 for all rows, which is the mean fare_amount from the training set.

```
import pandas as pd
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression, ElasticNet, Ridge,Lasso
from sklearn.metrics import root_mean_squared_error,r2_score
from sklearn.model_selection import train_test_split,KFold, RandomizedSearchCV
sns.set_theme(palette="viridis",rc={"figure.figsize": (8, 5)},style="darkgrid")
```

```
train = pd.read_csv('data.nosync/train.csv', nrows=1000000)
test = pd.read_csv('data.nosync/test.csv')
```

Data Cleaning & Feature Engineering

```
print("--- Missing Values Per Column ---")

print(train.isna().sum())

print("--- Data Types ---")
print(train.dtypes)
```

```
--- Missing Values Per Column ---
key                   0
fare_amount           0
pickup_datetime       0
pickup_longitude      0
pickup_latitude       0
dropoff_longitude    10
dropoff_latitude     10
passenger_count       0
dtype: int64
--- Data Types ---
key                     str
fare_amount         float64
pickup_datetime         str
pickup_longitude    float64
pickup_latitude     float64
dropoff_longitude   float64
dropoff_latitude    float64
passenger_count       int64
dtype: object
```

```
print("--- Descriptive Summary ---")
print(train.describe())
```

```
--- Descriptive Summary ---
          fare_amount  pickup_longitude  pickup_latitude  dropoff_longitude  \
count  1000000.000000    1000000.000000   1000000.000000      999990.000000
mean        11.348079        -72.526640        39.929008         -72.527860
std          9.822090         12.057937         7.626154          11.324494
min        -44.900000      -3377.680935     -3116.285383       -3383.296608
25%          6.000000        -73.992060        40.734965         -73.991385
```

```
50%          8.500000        -73.981792        40.752695         -73.980135
75%         12.500000        -73.967094        40.767154         -73.963654
max        500.000000       2522.271325      2621.628430          45.581619

       dropoff_latitude  passenger_count
count     999990.000000   1000000.000000
mean          39.919954         1.684924
std            8.201418         1.323911
min        -3114.338567         0.000000
25%           40.734046         1.000000
50%           40.753166         1.000000
75%           40.768129         2.000000
max         1651.553433       208.000000
```

```
upper_limit = train["fare_amount"].quantile(0.75)+1.5*stats.iqr(train["fare_amount"])
lower_limit = train["fare_amount"].quantile(0.25)-1.5*stats.iqr(train["fare_amount"])
outliers = (train["fare_amount"]<=lower_limit) & (train["fare_amount"]>=upper_limit)
print(f"""
----  Fare  Amount Analysis  ----

    Upper limit: {upper_limit},
    Lower limit: {lower_limit},
    Outliers percentage: {len(train[outliers])*100.00/len(train)},
    Negative price in : {len(train[train["fare_amount"]<0])} Rows
    """)
```

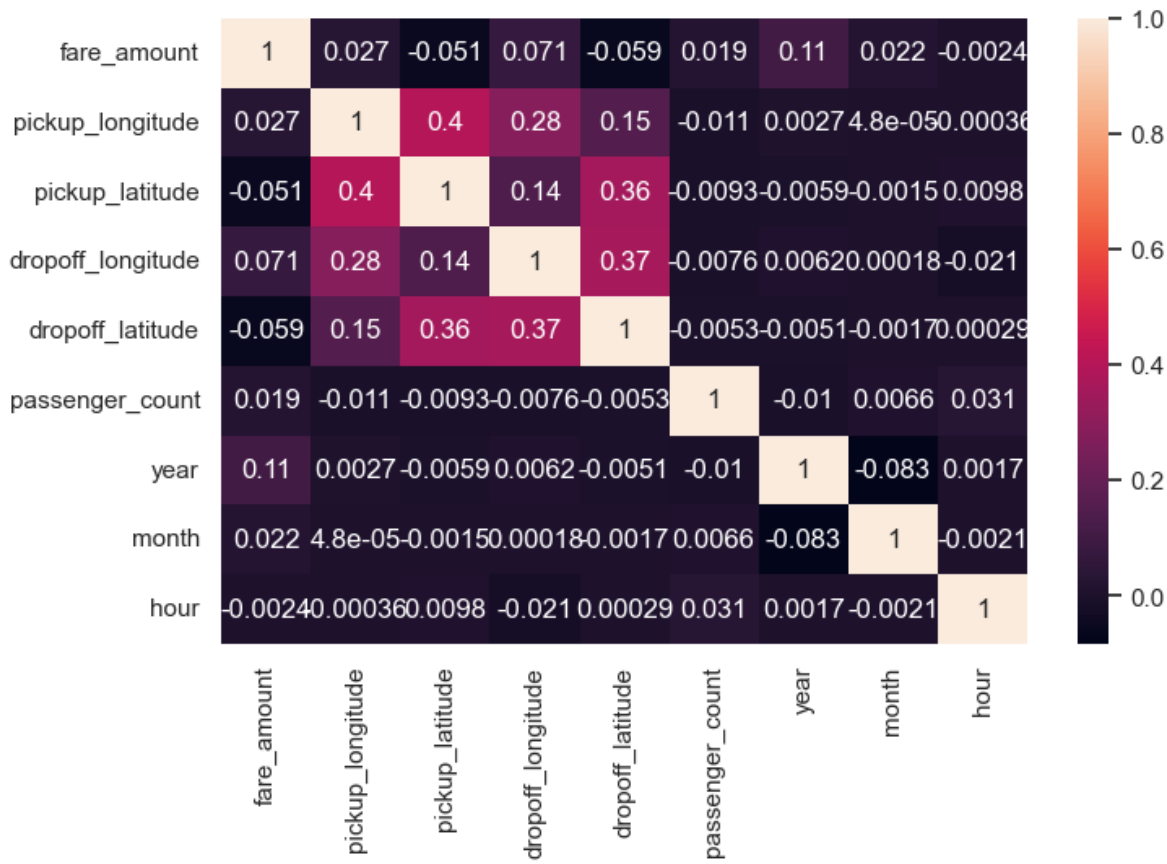
```
----  Fare  Amount Analysis  ----

    Upper limit: 22.25,
    Lower limit: -3.75,
    Outliers percentage: 0.0,
    Negative price in : 38 Rows
```

```
print("--- Removing negative 38 rows ---\n")
train_clean =train[train["fare_amount"]>=0]
train_clean["pickup_datetime"]=pd.to_datetime(train_clean["pickup_datetime"],errors="coerce")
train_clean["year"]=train_clean["pickup_datetime"].dt.year
train_clean["month"]=train_clean["pickup_datetime"].dt.month
train_clean["hour"]=train_clean["pickup_datetime"].dt.hour
```

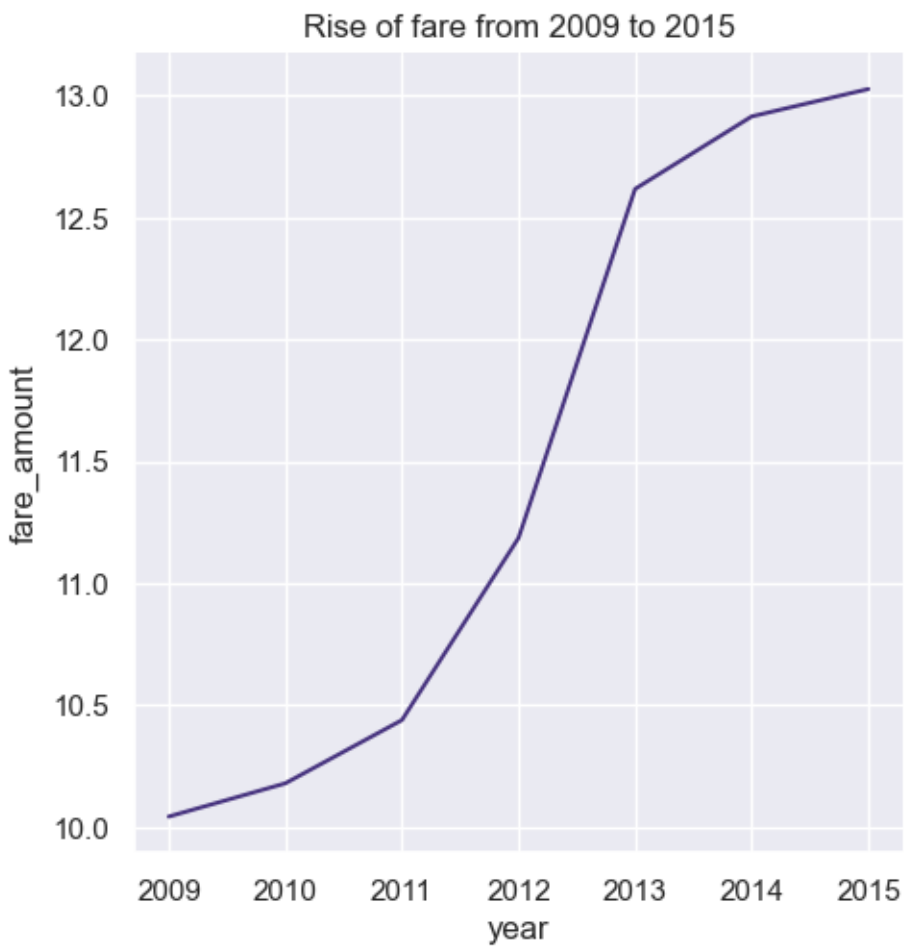
```
--- Removing negative 38 rows ---
```

Exploratory Data Analysis (EDA)

```
sns.heatmap(train_clean.corr(numeric_only=True,method="kendall"),annot=True)
plt.show()
```

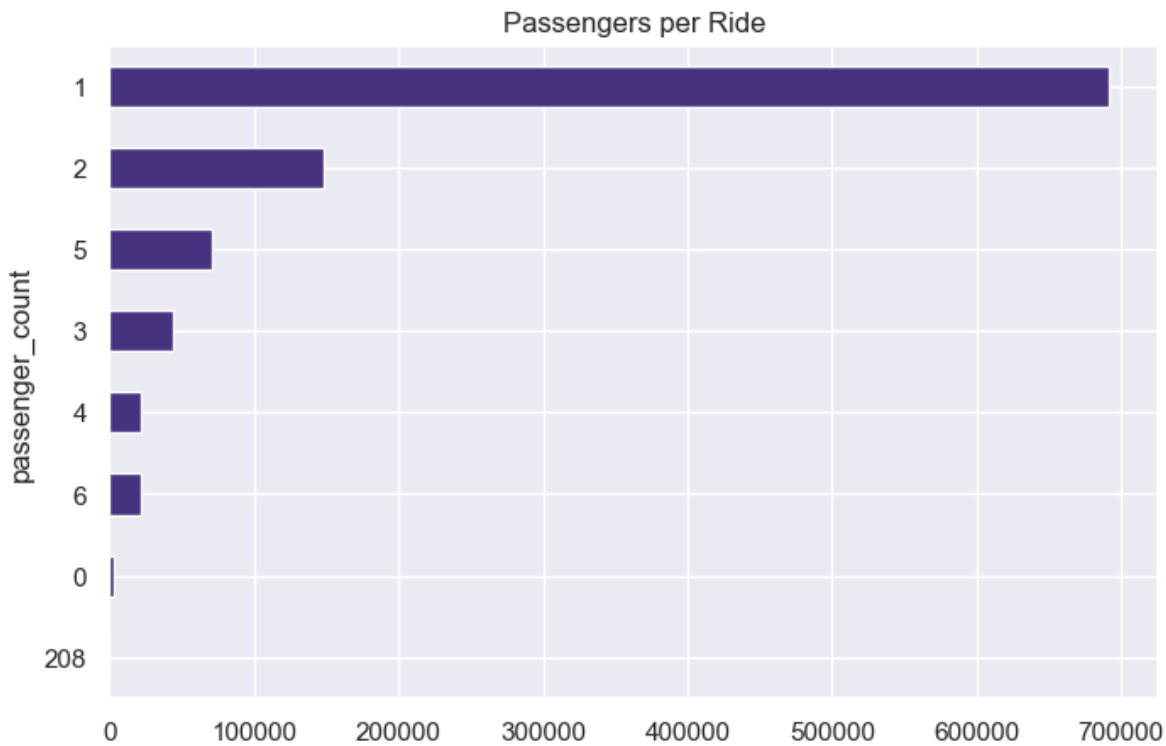



```
sns.relplot(x="year",y="fare_amount",data=train_clean,kind="line",errorbar=None)
plt.title("Rise of fare from 2009 to 2015")
plt.show()
```




```
sns.lineplot(x="hour",y="fare_amount",data=train_clean,estimator="median",marker="o",errorba
plt.xlabel('Hour')
plt.ylabel('Median fare amount')
plt.xticks(range(24))
plt.title('Fare amount based on time')
plt.show()
```

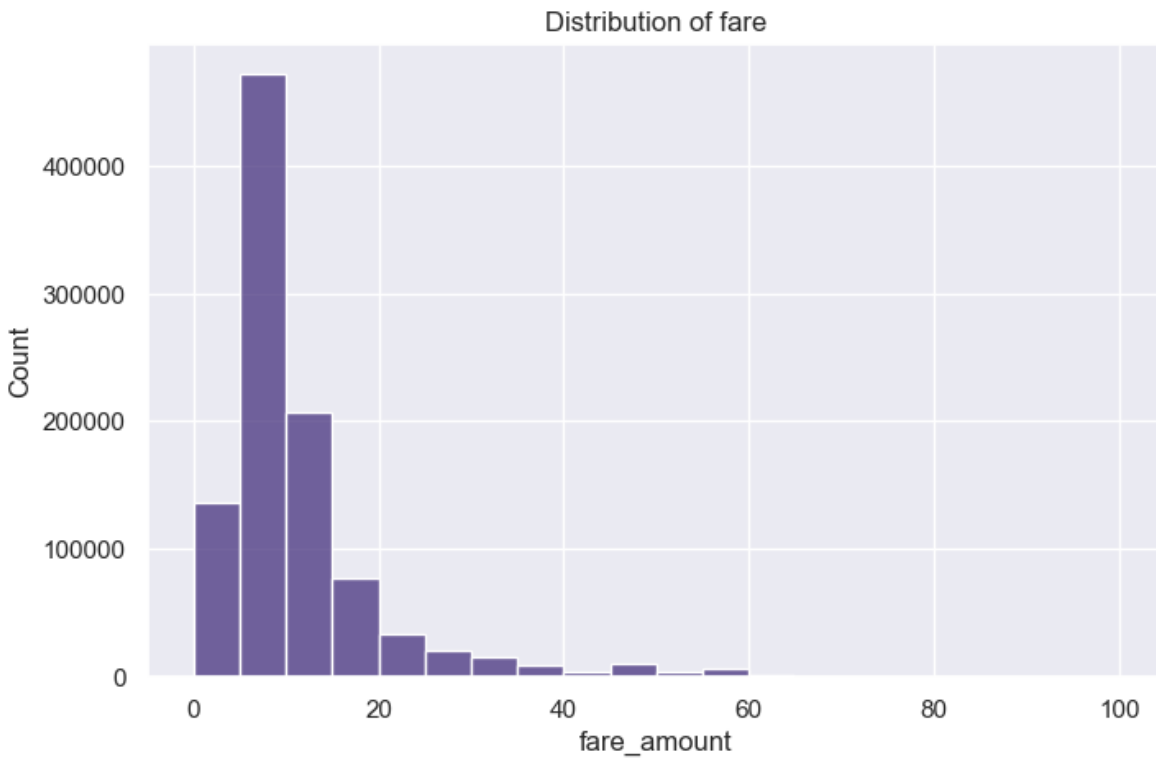
```
train_clean["passenger_count"].value_counts(ascending=True).plot(kind="barh")
plt.title("Passengers per Ride")
plt.show()
```




```
print(f"Number of rows with either 0 and one instance of 208 passengers are : \
{len(train_clean[train_clean['passenger_count'].isin([208,0])])}\n")

print("--- Removing thses outliers ---\n")
train_clean=train_clean[~train_clean["passenger_count"].isin([208,0])]
```

```
Number of rows with either 0 and one instance of 208 passengers are : 3566

--- Removing thses outliers ---
```

```
sns.histplot(x="fare_amount",bins=np.linspace(0,100,21),data=train_clean)
plt.title("Distribution of fare")
plt.show()
```




```
def haversign_distance(lat1,lat2,lon1,lon2):
    """
    d = 2R · arcsin( √[ sin²((lat  - lat )/2) + cos(lat )·cos(lat )·sin²((lon  - lon )/2) ] )
    6371 is earth radius in Km
    """
    lat1,lat2,lon1,lon2 = map(np.radians,[lat1,lat2,lon1,lon2])
    dlat = lat2-lat1
    dlon = lon2-lon1
    # Haversine formula
    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))
    return c* 6371
train_clean["distance_km"]= haversign_distance(train_clean['pickup_latitude'],
                                               train_clean['dropoff_latitude'],
                                               train_clean['pickup_longitude'],
                                               train_clean['dropoff_longitude'])
```

```
latitute_range = (40.5, 41.0)
longitude_range = (-74.3, -73.7)
coordinate_outliers = ~train_clean["pickup_latitude"].between(*latitute_range)| \
```

```
    ~train_clean['dropoff_latitude'].between(*latitute_range)  | \
    ~train_clean['pickup_longitude'].between(*longitude_range) | \
    ~train_clean['dropoff_longitude'].between(*longitude_range)
print(f"""
        Filtering the langitute between Staten Island to Yonkers
        and longitude between New Jersey to JFK/Queens.
        Number of rows affected: {len(train_clean[coordinate_outliers])}
       """)
print("--- Removing thses outliers ---\n")
train_clean =train_clean[~coordinate_outliers]
```

```
        Filtering the langitute between Staten Island to Yonkers
        and longitude between New Jersey to JFK/Queens.
        Number of rows affected: 21799

--- Removing thses outliers ---
```

Modelling

Base-line model

```
# 1. Feature selection
X=train_clean[["passenger_count","year","month","hour","distance_km"]]

# 2. Target selection
y=train_clean["fare_amount"]

# 3. 20% as test set/unseen data
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

# 4. Looping 3 linear model for comparison
models= {"Linear Regression": LinearRegression(),
         "Ridge Regression": Ridge(random_state=42),
         "ElasticNet": ElasticNet(random_state=42)}
for name,model in models.items():
    model.fit(X_train,y_train)
    y_pred=model.predict(X_test)
    rmse= root_mean_squared_error(y_test,y_pred)
```

```
    r2=r2_score(y_test,y_pred)
    print(f"Model: {name}\nRMSE: {rmse:.2f} R2: {r2:.2f}")
```

```
Model: Linear Regression
RMSE: 4.60 R2: 0.76
Model: Ridge Regression
RMSE: 4.60 R2: 0.76
Model: ElasticNet
RMSE: 4.64 R2: 0.76
```

Hyperparemeter tuninng with K-Fold cross validation for best baseline model

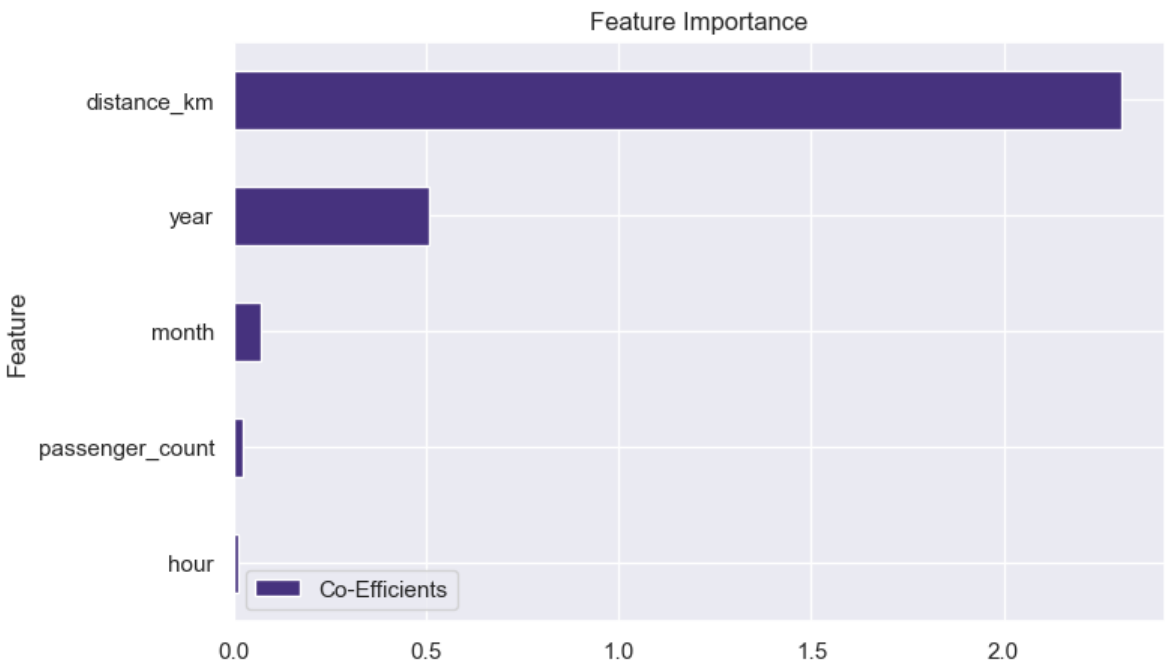
```
# 1. Cross validation with 5 split for robustness and to reduce biasness
kfold=KFold(n_splits=5,shuffle=True,random_state=42)

# 2. hyperparameter tuning of Ridge regression parameters: alpha and solver to penalize large
ridge=Ridge(random_state=42) # fixed seed for reproducibility
param_gid={"alpha":np.linspace(0.1,1,10),"solver":["saga","lsqr"]} #lsqr for faster iteratio

# 3. Randomized Seach of all 20 parameter combinations across 5 splits (total 100 fits)
ridge_gridcv=RandomizedSearchCV(ridge,param_gid,cv=kfold,n_iter=20)

# 4. Fitting and predicting using the best estimator and parameters
ridge_gridcv.fit(X_train,y_train)
y_pred=ridge_gridcv.predict(X_test)

# 5. Results
rmse = root_mean_squared_error(y_test, y_pred)
print(f"Tuned Parameters for the best estimator:{ridge_gridcv.best_params_}\nMean Score of t
print(f"Local Validation RMSE: {rmse:.2f}\nR2 Score: {ridge_gridcv.score(X_test,y_test):.2f}
```

```
Tuned Parameters for the best estimator:{'solver': 'saga', 'alpha': np.float64(0.1)}
Mean Score of the best estimator:0.7503253324975215
Local Validation RMSE: 4.60
R2 Score: 0.76
```

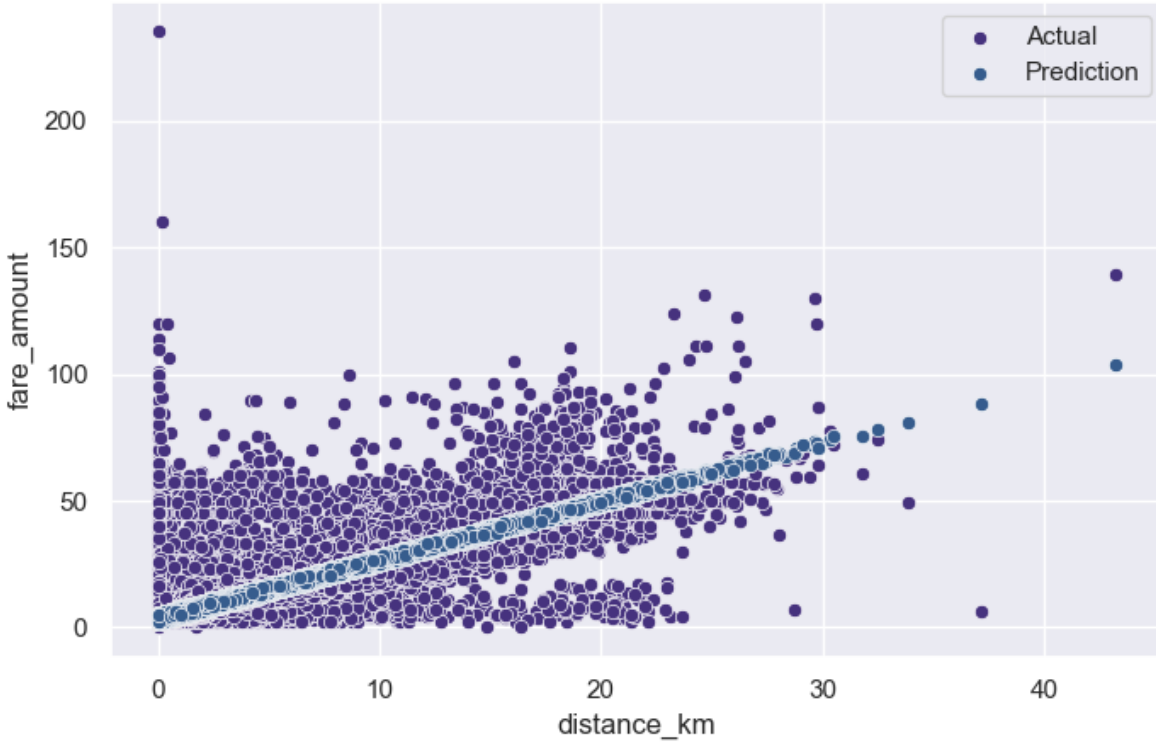
- The RMSE of 4.60 means on average predictions are off by $4.60.
- R2 Score of 0.76 means, the seleted features explain 76% of the variance in the fare amount.

Feature Importance

```
pd.DataFrame({"Feature":X.columns,"Co-Efficients":ridge_gridcv.best_estimator_.coef_}).sort_v
    .plot.barh("Feature","Co-Efficients")
plt.title("Feature Importance")
plt.xticks()
plt.show()
```




```
sns.scatterplot(x=X_test["distance_km"],y=y_test,label="Actual")
sns.scatterplot(x=X_test["distance_km"],y=y_pred,label="Prediction")
plt.show()
```



- Scatter plot of distance vs fare explained linear model is not suitable for the dataset as the predicted value is like mean line.

Kaggle Submission

```
model.fit(X,y=train_clean["fare_amount"])
test['pickup_datetime'] = pd.to_datetime(test['pickup_datetime'])
test['year'] = test['pickup_datetime'].dt.year
test['month'] = test['pickup_datetime'].dt.month
test['hour'] = test['pickup_datetime'].dt.hour
test["distance_km"]=haversign_distance(test['pickup_latitude'],
                                        test['dropoff_latitude'],
                                        test['pickup_longitude'],
                                        test['dropoff_longitude'])
latitute_range = (40.5, 41.0)
longitude_range = (-74.3, -73.7)
coordinate_outliers = ~test["pickup_latitude"].between(*latitute_range)| \
    ~test['dropoff_latitude'].between(*latitute_range)  | \
    ~test['pickup_longitude'].between(*longitude_range) | \
```

```
    ~test['dropoff_longitude'].between(*longitude_range)

test_features = test[["passenger_count", "year", "month", "hour","distance_km"]]
test["fare_amount"] = ridge_gridcv.predict(test_features)
test[["key","fare_amount"]].to_csv('submission.csv', index=False)
```